

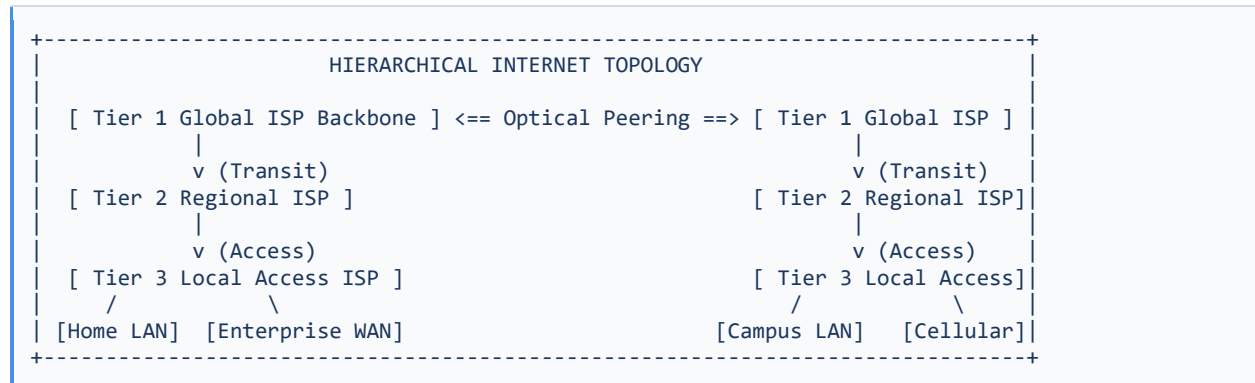
WEB TECHNOLOGIES

UNIT I

1. Introduction to Internet

Definition: The Internet is a globally interconnected system of computer networks that communicate using the standardized Transmission Control Protocol / Internet Protocol (TCP/IP) suite.

Basic Concept: The Internet is an open, packet-switched network of networks connecting billions of private, public, academic, business, and government computing devices across local to global scopes.



The physical architecture comprises three foundational tiers: Edge hosts (client endpoints and servers), Core packet switches (routers and switches), and Access networks (fiber-optic undersea lines, coaxial lines, satellite links, and cellular radio towers).

How the Internet Works: Step-by-step data path from sender to receiver:

- **1. Data Segmentation:** Application data is segmented into packets at the transport/network layer.
- **2. Encapsulation:** Packets are encapsulated with source/destination IP and MAC addresses.
- **3. Dynamic Routing:** Intermediate routers inspect destination IP addresses, consult routing tables (BGP/OSPF), and forward packets across optimal links.
- **4. Media Multiplexing:** Statistical multiplexing enables packets from multiple sources to share transmission media dynamically.
- **5. Reassembly:** The receiving endpoint validates checksums, reassembles packets in order, and delivers payload to the application.

Major Features: Key Characteristics:

- **Decentralized Governance:** Operates without a single centralized governing authority.
- **Packet Switching:** Maximizes bandwidth efficiency via on-demand line sharing.
- **Platform Independence:** Enables diverse hardware and OS platforms to communicate seamlessly.
- **Scalability:** Supports continuous network growth without operational collapse.
- **Fault Tolerance:** Dynamic rerouting maintains communication despite intermediate node failures.

Exam Focus: Introduction to Internet

- The Internet operates strictly on Packet Switching (dynamic line sharing), not Circuit Switching (dedicated line allocation).
- The Internet is structured in a 3-tier ISP hierarchy: Tier 1 (Global Backbones), Tier 2 (Regional Transit), and Tier 3 (Local Access).

2. History of the Internet

Historical Evolution: The evolution of the Internet spans military research, academic standardization, and commercial globalization.

```
[1969] ARPANET Founded (First 4-node packet-switching network: UCLA, SRI, UCSB, Utah)
      |
      v
[1983] TCP/IP Standardization ("Flag Day", Jan 1 - Replaced NCP; official Internet birth)
      |
      v
[1984] Domain Name System (DNS) created by Paul Mockapetris
      |
      v
[1986] NSFNET established (High-speed academic supercomputing backbone)
      |
      v
[1989-1991] Tim Berners-Lee invents World Wide Web (HTML, HTTP, URL) at CERN
      |
      v
[1995] NSFNET decommissioned; full commercialization and privatization of backbones
      |
      v
[2000s-Present] Mobile Broadband (4G/5G), Cloud Computing, and Internet of Things (IoT)
```

- **ARPANET (1969):** Commissioned by DARPA to create a survivable distributed communications network. First message transmitted between UCLA and SRI in October 1969.
- **TCP/IP Adoption (1983):** Vint Cerf and Bob Kahn designed TCP/IP to replace the legacy Network Control Program (NCP). Standardized on January 1, 1983.
- **NSFNET (1986):** Provided an upgraded academic backbone connecting universities and national laboratories.
- **World Wide Web (1989-1991):** Invented by Sir Tim Berners-Lee at CERN to allow scientists to share documents using hypertext.
- **Commercial Era (1995):** Public Internet access commercialized with private ISPs and the release of graphical browsers (Mosaic, Netscape).

Exam Focus: Internet History

- Father of the Internet: Vinton Cerf and Robert Kahn (designed TCP/IP).
- Inventor of the World Wide Web: Sir Tim Berners-Lee (CERN, 1989).
- Official technical birth date of the Internet: January 1, 1983 (TCP/IP Flag Day).

3. Internet Design Principles

Architectural Overview: The technical architecture of the Internet is governed by fundamental engineering principles defined in RFC 1958 and RFC 3439.

- **1. Packet Switching:** Data is partitioned into individual packets that travel independently across shared lines, maximizing utilization efficiency.
- **2. End-to-End Principle:** Application intelligence and state maintenance reside exclusively at the endpoint hosts. The core network remains simple, fast, and unaware of application logic.
- **3. Layered Architecture:** Functionality is divided into modular protocol layers (TCP/IP stack). Modifying one layer requires zero changes to adjacent layers.
- **4. Decentralization:** The network has no single point of failure; routing algorithms dynamically bypass damaged links.

- **5. Open Standards:** Protocols are vendor-neutral, publicly documented, and managed by open international bodies (IETF, W3C).
- **6. Robustness Principle:** Postel's Law (RFC 793): 'Be conservative in what you do, be liberal in what you accept from others.' Ensures reliable inter-vendor operation.

4. Internet Protocols

Definition: A protocol is an agreed-upon set of formal syntax rules, message semantics, and synchronization behaviors governing data exchange over a network.

A. File Transfer Protocol (FTP)

FTP Overview: An application-layer client-server protocol used for transferring, managing, and manipulating files across TCP networks.

- **Dual-Channel Architecture:** FTP operates using two distinct, concurrent TCP connections (out-of-band signaling):
 - **Control Connection (Port 21):** Maintained for the entire session; transmits user credentials, navigation commands (CWD, PWD), transfer commands (RETR, STOR), and status reply codes.
 - **Data Connection (Port 20):** Created dynamically when a file transfer or directory listing is requested, and closed immediately upon completion.
- **Active Mode:** Client sends PORT command specifying client port; server initiates connection from Port 20 to client (often blocked by client firewalls).
- **Passive Mode (PASV):** Client sends PASV command; server opens a random high-numbered port and sends it to the client; client initiates connection (firewall-friendly).
- **Security Concern:** Plaintext transmission of credentials and data creates security vulnerabilities. Replaced by SFTP (SSH File Transfer, Port 22) and FTPS (FTP over TLS, Port 990/21).

B. TCP/IP Protocol Suite

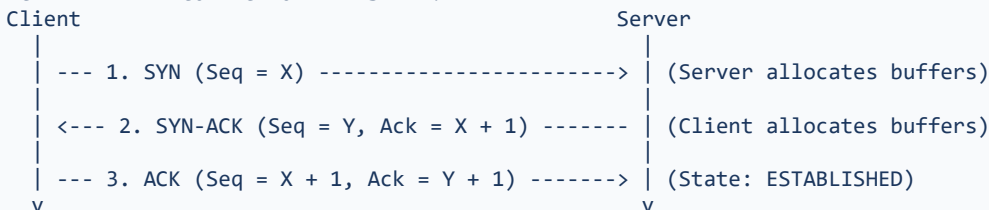
Overview: The foundational hierarchical communication model powering the Internet, organized into 4 functional layers.

TCP/IP Layer	Primary Responsibilities	Associated Protocols
4. Application	Direct network interface for user applications; formatting and presentation.	HTTP, HTTPS, FTP, SMTP, DNS, Telnet, SSH, SNMP
3. Transport	End-to-end process-to-process delivery, port addressing, reliability, flow control.	TCP (Connection-Oriented), UDP (Connectionless)
2. Internet	Logical host addressing (IP), datagram encapsulation, routing across subnets.	IPv4, IPv6, ICMP, ARP, IGMP, BGP, OSPF
1. Network Access	Physical hardware interfacing, MAC framing, electrical/optical signal transmission.	Ethernet (IEEE 802.3), Wi-Fi (802.11), PPP, Fiber

Transport Layer Analysis: Comparison between TCP and UDP transport protocols:

Feature	TCP (Transmission Control Protocol)	UDP (User Datagram Protocol)
Connection Model	Connection-Oriented (3-Way Handshake)	Connectionless (No connection setup)
Reliability	Guaranteed delivery with retransmissions	Best-effort; dropped packets are not recovered
Data Ordering	Guarantees ordered byte stream arrival	Packets may arrive out-of-order
Header Size	20 to 60 Bytes	Fixed at 8 Bytes
Flow & Congestion	Sliding window flow control; AIMD congestion	None (Sender transmits at line speed)
Target Applications	Web (HTTP), Email (SMTP), File Transfer (FTP)	Video Streaming, VoIP, DNS queries, Gaming

TCP THREE-WAY CONNECTION HANDSHAKE:



C. Simple Mail Transfer Protocol (SMTP)

SMTP Overview: A push-based, text-oriented application-layer protocol used to transmit electronic mail from clients to mail servers and between relay mail servers.

- **1. Mail Submission:** Sender composes email in Mail User Agent (MUA) and pushes to sending Mail Transfer Agent (MTA) via SMTP.
- **2. Domain Resolution:** Sender MTA extracts destination domain and queries DNS for the MX (Mail Exchanger) record to locate the target server IP.
- **3. Server Relay:** Sender MTA connects to recipient MTA on Port 25 via SMTP commands (HELO/EHLO, MAIL FROM, RCPT TO, DATA, QUIT).
- **4. Mail Retrieval:** Recipient uses POP3 (Port 995/110) or IMAP (Port 993/143) to retrieve stored messages from mailbox.
- **Standard Ports:** Port 25 (Server-to-Server Relay), Port 587 (Client Mail Submission with TLS), Port 465 (SMTPS over SSL).

Protocol	Role / Direction	Storage Behavior	Default Secure Port
SMTP	Push (Client -> Server -> Server)	Transit protocol (No storage)	Port 587 (Submission) / 25
POP3	Pull (Server -> Client)	Downloads locally; deletes on server	Port 995 (POP3S) / 110
IMAP	Sync (Server <-> Client)	Leaves mail on server; syncs folders across devices	Port 993 (IMAPS) / 143

D. Telnet vs. SSH

Telnet Overview: An application-layer protocol used to provide bidirectional, interactive terminal access to remote hosts over Port 23.

Security Vulnerability: Telnet sends all keystrokes, usernames, and passwords in clear, unencrypted plaintext across the wire, making it vulnerable to packet interception. It has been replaced by Secure Shell (SSH) over Port 22.

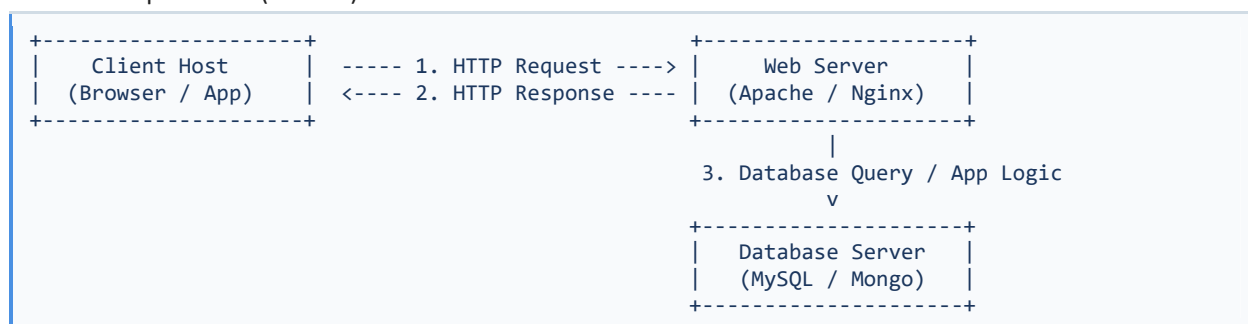
Feature	Telnet	SSH (Secure Shell)
Port	Port 23	Port 22
Security / Encryption	None (Plaintext transmission)	Strong cryptographic encryption (AES, RSA)
Authentication	Plaintext passwords	Public-key cryptography, certificates, passwords
Data Integrity	No verification against tampering	Cryptographic HMAC ensures integrity
Current Usage	Obsolete for public networks; test labs only	Universal industry standard for remote administration

Exam Focus: Protocols

- FTP uses Port 21 for Control and Port 20 for Data (Dual-connection architecture).
- SMTP is a Push protocol; POP3 and IMAP are Pull/Retrieval protocols.
- Telnet (Port 23) is insecure plaintext; SSH (Port 22) is cryptographically secure.

5. Client-Server Communication

Definition: A distributed computing architecture that partitions tasks between service requesters (clients) and service providers (servers).



- **Client:** Initiates communication by sending structured requests over network protocols.
- **Server:** Passively listens on dedicated network ports, processes requests, enforces security/business logic, and returns responses.
- **Advantages:** Centralized data control, simplified backup management, scalable resources, robust access security.
- **Limitations:** Potential single point of failure without clustering; server congestion during traffic spikes; infrastructure hosting costs.

6. The Web (World Wide Web)

Definition: A global hypermedia information space where resources are identified by Uniform Resource Locators (URLs), interlinked by hypertext links, and transferred via HTTP/HTTPS protocols.

Relationship: The Internet is the underlying physical network highway (cables, routers, TCP/IP). The World Wide Web is an application service that travels across that highway.

Parameter	Internet	World Wide Web (WWW)
Nature	Global physical network infrastructure	Information sharing application service
Inception	1969 (ARPANET) / 1983 (TCP/IP)	1989 (Invented by Tim Berners-Lee)
Governing Protocols	IP, TCP, UDP, BGP, ARP, ICMP	HTTP, HTTPS, WebSocket
Addressing	IP Addresses (e.g., 192.0.2.1)	URLs / URIs (e.g., https://example.com)
Core Medium	Data packets, frames, hardware links	HTML documents, CSS stylesheets, media files

Important Definitions — Unit I

- Internet: A global network of interconnected computer networks communicating via TCP/IP.
- Packet Switching: Dividing data streams into addressed packets that traverse shared paths independently.
- End-to-End Principle: Network architectural design placing application logic at hosts while keeping core routers simple.
- TCP: Reliable, connection-oriented, sequenced transport protocol using a 3-way handshake.
- IP: Connectionless, best-effort network layer protocol handling logical addressing and packet routing.
- FTP: Application protocol using dual channels (Port 21 control, Port 20 data) for file transfer.
- SMTP: Push-based application protocol for transmitting electronic mail over Port 25/587.
- SSH: Cryptographic network protocol for secure remote command-line login over Port 22.
- Client-Server Model: Distributed architecture where client programs request services and servers fulfill them.

Quick Revision — Unit I

- ARPANET (1969) transitioned to TCP/IP on January 1, 1983 (Flag Day).
- TCP/IP 4 Layers: Application -> Transport -> Internet -> Network Access.
- TCP provides reliability, sequencing, and flow control; UDP provides low-latency best-effort delivery.
- FTP uses Port 21 (commands) and Port 20 (data); Passive mode is firewall-friendly.
- SMTP sends mail; POP3/IMAP retrieve mail from mail servers.
- The Internet is the physical network infrastructure; the Web is an application operating over it.

UNIT II

1. Introduction to World Wide Web

Definition: The World Wide Web (WWW) is a distributed hypermedia information system created in 1989 by Sir Tim Berners-Lee at CERN.

Three Pillars of the Web: The WWW operates on three fundamental pillars:

- **1. HTML (Hypertext Markup Language):** Standard structural language used to format web documents.
- **2. HTTP (Hypertext Transfer Protocol):** Stateless client-server application protocol governing web message exchanges.
- **3. URL / URI (Uniform Resource Locator):** Standardized universal addressing system pinpointing resource locations.

2. Evolution of the Web (Web 1.0 vs. Web 2.0 vs. Web 3.0)

Generational Overview: The Web has progressed through three structural generations:

Feature	Web 1.0 (1990 - 2004)	Web 2.0 (2004 - Present)	Web 3.0 (Emerging Era)
Core Nature	Read-Only Web	Read-Write (Participatory) Web	Read-Write-Trust (Semantic Web)
User Role	Passive content consumer	Active creator and collaborator	Data owner and agent collaborator
Content Source	Webmaster published	User-Generated Content (UGC)	Semantic data graphs, AI synthesized
Key Technologies	Static HTML, basic CGI, tables	HTML5, AJAX, REST APIs, JS Frameworks	RDF/OWL, SPARQL, AI/LLMs, Blockchains
Architecture	Static Web Server -> Client	Dynamic Web + Cloud + Database	Decentralized Nodes, Edge Computing
Representative Sites	Personal homepages, Britannica	Wikipedia, YouTube, Facebook, Twitter	dApps, Solid Pods, Wolfram Alpha

3. Static and Dynamic Websites

Website Classifications: Comparison and operational mechanics of static versus dynamic web architecture.

STATIC WORKFLOW:

Browser == 1. GET /page.html ==> Web Server == 2. Fetches from Disk ==> Returns HTML

DYNAMIC WORKFLOW:

Browser == 1. GET /profile?id=5 ==> Web Server == 2. Executes Script (Node/PHP)

3. Queries Database (SQL)

Browser <== 5. Returns Generated HTML <== 4. Returns Data <== [Database]

Feature	Static Websites	Dynamic Websites
Content Delivery	Pre-written, fixed files stored on server disk	Generated in real time per user request
Server-Side Logic	None; server retrieves files directly from disk	Executes scripts (Node.js, Python, PHP, Java)
Database Integration	No database required	Relies heavily on databases (SQL / NoSQL)
Performance / Speed	Very fast (cache-friendly, CDN ready)	Slower due to script execution & DB queries
Maintenance	Tedious; updating layout requires editing all files	Easy; templates populate data dynamically
Primary Use Cases	Brochure sites, documentation, portfolios	E-commerce, social networks, web banking

4. Web Applications

Definition: A Web Application is an interactive software program hosted on remote servers and delivered across the Internet through a web browser interface.

```
THREE-TIER WEB APPLICATION ARCHITECTURE:  
[ Tier 1: Presentation Layer ] --> Browser UI (HTML5, CSS3, JavaScript, React)  
    | (HTTP / REST API)  
    v  
[ Tier 2: Application Logic ] --> Server Runtime (Node.js, Django, Spring, .NET)  
    | (Database Driver / SQL)  
    v  
[ Tier 3: Data Storage Tier ] --> Database Engine (PostgreSQL, MySQL, MongoDB)
```

Parameter	Informational Website	Web Application
Primary Goal	Present information to be read/viewed	Allow users to perform tasks and process data
Interactivity	Low (clicking links, viewing images)	High (CRUD data operations, dynamic state)
Authentication	Generally not required	Mandatory user authentication & session state
Examples	Wikipedia, news portal, university site	Google Docs, Trello, Canva, NetBanking

5. Web Development Technologies (HTML, CSS, JS, XML)

- **HTML:** Hypertext Markup Language. Defines content structure and document semantics ('The Skeleton').
- **CSS:** Cascading Style Sheets. Controls visual presentation, typography, colors, and responsive layouts ('The Skin / Clothing').
- **JavaScript:** Client-side programming language enabling dynamic behaviors, user event handling, and asynchronous requests ('The Muscles / Behavior').

- **XML:** Extensible Markup Language. Self-descriptive markup language designed for transporting and storing structured data across heterogeneous platforms.

Feature	HTML (HyperText Markup Language)	XML (Extensible Markup Language)
Primary Purpose	Display data and format web page layout	Store, describe, and transport data
Tags Definition	Predefined standard tags (<h1>, <p>, <a>)	Custom, user-defined tags (<student>, <gpa>)
Syntax Strictness	Lenient (Browsers correct minor syntax bugs)	Strict (Malformed syntax halts parsing)
Case Sensitivity	Case-insensitive (<h1> matches <H1>)	Strictly case-sensitive (<Book> != <book>)
Closing Tags	Optional in void elements (,)	Mandatory for every single element

6. Web Protocols: HTTP and HTTPS

HTTP Overview: Hypertext Transfer Protocol is a stateless, application-layer request-response protocol operating over Port 80 by default.

HTTP Methods: Standard Request Methods (Verbs):

- **GET:** Retrieves resource data from server without modifying server state (Safe & Idempotent). Data passed in URL query string.
- **POST:** Submits payload data to server for resource creation or processing. Data enclosed in request body.
- **PUT:** Replaces entire target resource with uploaded payload (Idempotent).
- **PATCH:** Applies partial modifications to existing resource.
- **DELETE:** Removes the specified resource from server (Idempotent).
- **HEAD:** Identical to GET, but server returns headers only (no response body).
- **OPTIONS:** Returns allowed HTTP communication methods for target URL.

HTTP Status Codes: Standard Status Code Categories:

Code Class	Meaning	Representative Examples
1xx Informational	Request received; continuing process	100 Continue
2xx Success	Action successfully received and accepted	200 OK, 201 Created, 204 No Content
3xx Redirection	Further action required to complete request	301 Moved Permanently, 304 Not Modified
4xx Client Error	Request contains bad syntax or unauthorized access	400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found
5xx Server Error	Server failed to fulfill valid request	500 Internal Server Error, 502 Bad Gateway, 503 Service Unavailable

HTTPS (HTTP Secure)

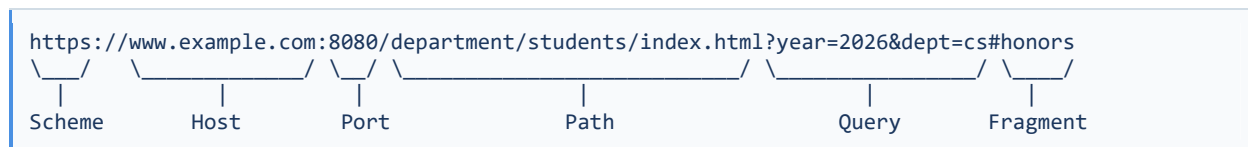
Definition: HTTPS encrypts HTTP communications by layering it over Transport Layer Security (TLS/SSL) cryptographic protocols operating over Port 443.

- **1. Encryption (Confidentiality):** Symmetric encryption (AES) obfuscates traffic, preventing network eavesdropping.
- **2. Authentication:** Digital X.509 certificates issued by trusted Certificate Authorities verify server identity.
- **3. Data Integrity:** Cryptographic hashes (HMAC) guarantee data cannot be altered or corrupted in transit.

Feature	HTTP	HTTPS
Protocol Structure	Plaintext HTTP over TCP	HTTP layered over TLS/SSL encryption over TCP
Default Port	Port 80	Port 443
Security Level	Insecure (Vulnerable to MITM attacks)	Highly secure (Encrypted end-to-end)
Digital Certificate	No certificate required	Mandatory SSL/TLS Certificate from trusted CA
Browser Indicator	Flagged as 'Not Secure'	Padlock icon displayed in address bar

7. URL (Uniform Resource Locator)

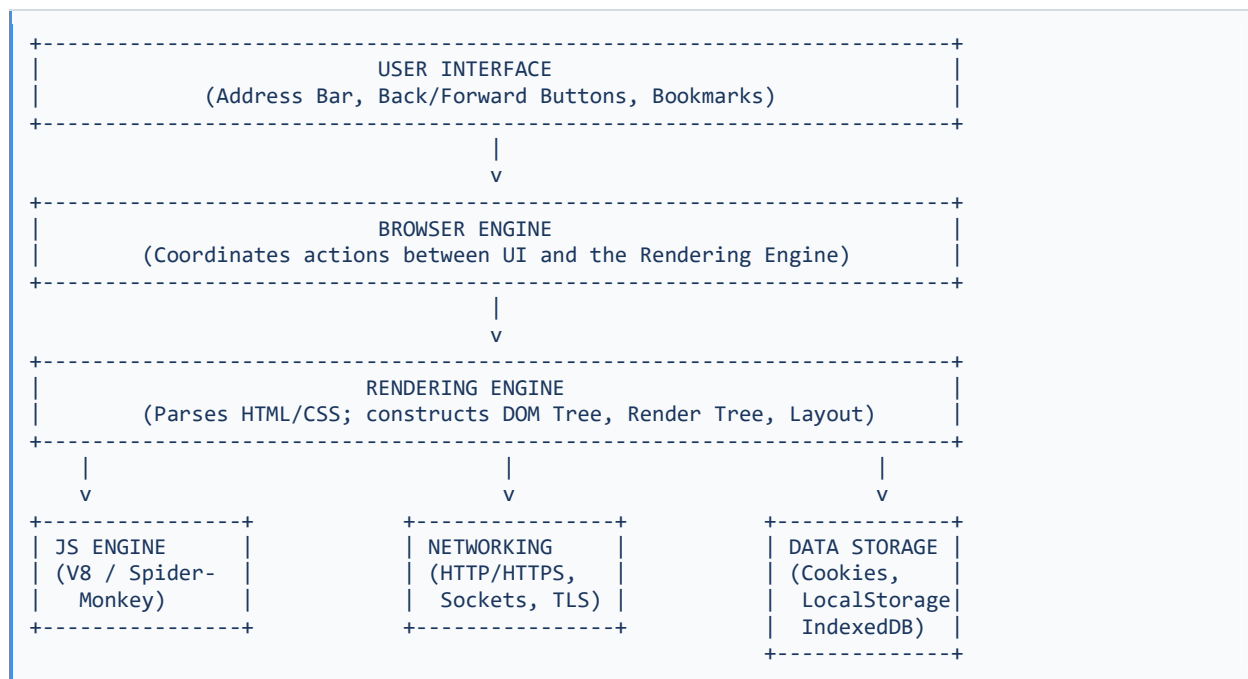
Definition: A standardized global string identifying the network location of a resource on the Internet and the protocol used to access it.



- **Scheme / Protocol:** Specifies access protocol (http, https, ftp, mailto).
- **Domain / Host:** Domain name or numeric IP address identifying the host server.
- **Port Number:** Technical port endpoint on server (Defaults: 80 for HTTP, 443 for HTTPS; omitted if standard).
- **Resource Path:** Hierarchical directory path pointing to specific resource on server.
- **Query String:** Key-value parameter string preceded by '?' and separated by '&' passed to server scripts.
- **Fragment Identifier:** Internal bookmark preceded by '#' pointing to specific element ID on page (Never sent to server).

8. Web Browser

Definition: Client software application that fetches, parses, executes, and visually renders resources from the World Wide Web.



Rendering Pipeline: Page Rendering Sequence: HTML parsed to DOM Tree -> CSS parsed to CSSOM Tree -> Combined into Render Tree -> Layout / Reflow geometry calculated -> Pixels painted to display screen.

Rendering Engines: Modern Browser Engines: Blink (Google Chrome, Microsoft Edge, Opera), Gecko (Mozilla Firefox), WebKit (Apple Safari).

9. Web Server

Definition: A server hardware host and software daemon that listens on designated ports (80/443), processes incoming HTTP/HTTPS requests, and delivers web content to clients.

Popular Servers: Primary software implementations: Apache HTTP Server (Modular, process/worker architecture), Nginx (Event-driven asynchronous architecture), Microsoft IIS (Windows Server integrated), Caddy (Automated HTTPS).

Feature	Web Browser	Web Server
Architecture Role	Client (Service Consumer)	Server (Service Provider)
Execution Model	Runs locally on end-user machines	Runs on remote dedicated servers/cloud
Operation	Active (Initiates outgoing HTTP requests)	Passive (Listens continuously on ports 80/443)
Primary Function	Parses HTML/CSS and paints GUI screen	Processes requests and delivers files/data

Examples

Google Chrome, Mozilla Firefox,
SafariApache HTTP Server, Nginx,
Microsoft IIS

10. Web Services

Definition: A standardized software system designed to support interoperable machine-to-machine application interaction across a network using open web protocols.

Feature	SOAP (Simple Object Access Protocol)	REST (Representational State Transfer)
Design Type	Strict formal protocol with rigid specifications	Flexible architectural style
Data Format	Exclusively XML	Multiple formats: JSON (Standard), XML, Text
Transport	Can run over HTTP, SMTP, TCP, JMS	Relies strictly on HTTP / HTTPS
Performance	Heavier payload; higher CPU parsing cost	Lightweight; fast serialization; bandwidth efficient
State Handling	Can be stateful or stateless	Strictly Stateless
Target Environment	Enterprise banking, legacy telecommunications	Modern web APIs, mobile backends, microservices

Important Definitions — Unit II

- World Wide Web: An information space of interconnected hypermedia documents accessed via URLs and HTTP.
- Web 1.0 / 2.0 / 3.0: Generations progressing from Read-Only (Static) -> Read-Write (Social/AJAX) -> Read-Write-Trust (Semantic/AI).
- Static Website: Fixed files delivered directly from server disk without dynamic computation.
- Dynamic Website: Pages constructed at runtime via server scripts and database queries.
- Web Application: Interactive multi-tier software executed in a browser (Presentation, Logic, Database tiers).
- HTTP: Stateless application protocol for transmitting hypermedia resources over Port 80.
- HTTPS: HTTP wrapped in TLS/SSL encryption for confidentiality, integrity, and authentication over Port 443.
- URL: Uniform Resource Locator specifying resource address and access protocol.
- Web Browser: Client software parsing HTML/CSS and executing JavaScript to render web pages.
- Web Server: Server daemon listening on ports 80/443 delivering web assets to clients.
- REST: Lightweight stateless web service architectural style utilizing standard HTTP verbs and JSON.

Quick Revision — Unit II

- Three Pillars of Web: HTML (Structure), HTTP (Protocol), URL (Addressing).
- HTTP Methods: GET (Read), POST (Create), PUT (Replace), DELETE (Remove).
- Status Codes: 200 (OK), 301 (Redirect), 404 (Not Found), 500 (Server Error).

- URL Components: Scheme -> Host -> Port -> Path -> Query String -> Fragment.
- HTTPS provides Encryption, Server Authentication, and Data Integrity.
- Rendering Engine constructs DOM + CSSOM -> Render Tree -> Layout -> Paint.
- All Web Services are APIs, but not all APIs are Web Services (Web services operate over network protocols).

UNIT III

1. Introduction to HTML

Full Form & Definition: HyperText Markup Language is the universal standard markup language used to structure documents and media for display in web browsers.

- **Role in Web Development:** HTML defines structural semantics ('The Skeleton'), CSS manages styling ('The Skin'), and JavaScript handles behavior ('The Muscles').
- **Current Standard:** HTML5 is the modern living standard maintained by W3C and WHATWG, offering native multimedia, semantic elements, and mobile-ready form controls.

2. HTML Document Structure

Document Template: Standard structural skeleton of every valid HTML5 document:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>University Examination Notes</title>
</head>
<body>
  <h1>Introduction to Web Technologies</h1>
  <p>This is a complete HTML5 standard webpage.</p>
</body>
</html>
```

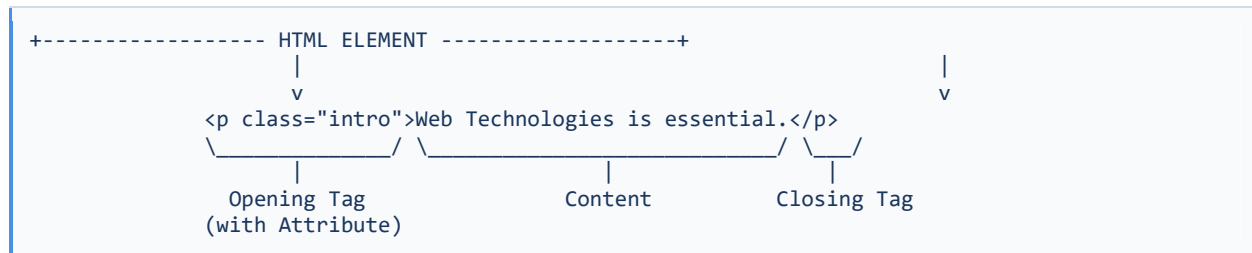
- **<!DOCTYPE html>:** Document Type Declaration informing browser to render in HTML5 standards mode.
- **<html lang='en'>:** Root container element wrapping all document code. lang='en' declares language.
- **<head>:** Container for document metadata, character sets, scripts, and stylesheets (Not directly visible).
- **<meta charset='UTF-8'>:** Declares UTF-8 character encoding supporting all global symbols.
- **<meta name='viewport' ...>:** Configures viewport scaling for responsive mobile displays.
- **<title>:** Defines text displayed on the browser window tab and search listings.
- **<body>:** Container for all user-visible page content (Headings, paragraphs, media, tables, forms).

3. HTML Editors

- **Text / Code Editors:** Source-code editors for writing raw markup manually with syntax highlighting, autocompletion, and linting. Examples: Visual Studio Code, Sublime Text, Notepad++.
- **WYSIWYG Editors:** 'What You See Is What You Get' visual design editors generating HTML in the background. Examples: Adobe Dreamweaver, Webflow.

- **Standard Workflow:** Write HTML code -> Save file with .html or .htm extension in UTF-8 -> Open in browser to render.

4. HTML Elements, Tags, and Attributes



Concept	Definition	Code Example
HTML Tag	Bracketed markup keyword indicating start/end of an element.	<code><p></code> and <code></p></code>
HTML Element	The complete unit: Opening tag + attributes + content + closing tag.	<code><p class='main'>Hello World</p></code>
Empty / Void Element	Elements that cannot contain content and have no closing tag.	<code>
</code> , <code><hr></code> , <code></code> , <code><input></code> , <code><meta></code>
HTML Attribute	Property modifier inside opening tag configuring element settings.	<code>class='main'</code> , <code>id='header'</code> , <code>src='pic.jpg'</code>

Global Attributes: Key Global Attributes:

- **id:** Unique identifier per page. Used for CSS styling, JS DOM manipulation, and internal anchor bookmarks.
- **class:** Non-unique classification identifier. Multiple elements can share identical class names for CSS grouping.
- **style:** Applies inline CSS styling directly on the element.
- **title:** Provides advisory tooltip text displayed on mouse hover.

5. Designing a Simple HTML Page

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Basic Formatting Demo</title>
</head>
<body>
  <!-- Heading hierarchy -->
  <h1>Main Title (H1)</h1>
  <h2>Section Title (H2)</h2>
  <h3>Subsection Title (H3)</h3>
```

```
<!-- Paragraphs and Line Breaks -->
<p>This is a paragraph of text.<br>This line is broken onto a new line.</p>
<hr>
<!-- Semantic and Visual Formatting -->
<p><b>Bold visual text</b> vs <strong>Strong semantic importance</strong>.</p>
<p><i>Italic visual text</i> vs <em>Emphasized semantic text</em>.</p>
<p><u>Underlined</u> and <mark>Highlighted mark text</mark>.</p>
<p>Water: H<sub>2</sub>O (Subscript) and Math: E = mc<sup>2</sup> (Superscript).</p>
<p>Code snippet: <code>console.log('Hello');</code></p>
</body>
</html>
```

6. The HEAD Tag and 7. The BODY Tag

HEAD Tag Elements: Child elements permitted inside <head>: <title>, <meta>, <link rel='stylesheet' href='...'>, <style>, <script>, <base>.

BODY Tag Elements: HTML5 Semantic structural layout elements inside <body>:

- **<header>**: Introductory container for site title, logo, and top navigation.
- **<nav>**: Block enclosing primary navigation hyperlinks.
- **<main>**: Represents central unique content of document (Only one <main> per page).
- **<section>**: Thematic grouping of related content with a heading.
- **<article>**: Self-contained independent composition (e.g., blog post, news story).
- **<aside>**: Tangential sidebar content, callouts, or advertisement blocks.
- **<footer>**: Closing footer containing copyright notices, authorship, and sitemaps.

8. Anchor Tag (<a>)

Anchor Syntax: Creates hyperlinks for document navigation, downloadable assets, page jumps, and mail links.

```
<!-- 1. Absolute URL (External Link in New Tab) -->
<a href="https://www.google.com" target="_blank">Search Google</a>

<!-- 2. Relative URL (Internal Site Link) -->
<a href="about.html" target="_self">About Us</a>

<!-- 3. Internal Page Bookmark Jump -->
<a href="#syllabus-section">Jump to Syllabus</a>
<!-- ... content ... -->
<h2 id="syllabus-section">Syllabus Heading</h2>

<!-- 4. Email and Telephone Links -->
<a href="mailto:admin@university.edu?subject=Inquiry">Email University</a>
<a href="tel:+1234567890">Call Support</a>
```

Target Value	Behavior
_self (Default)	Opens linked document in the same browsing tab/window
_blank	Opens linked document in a brand-new tab or window

_parent	Opens linked document in parent frame set
_top	Opens linked document in full body of current window, canceling frames

9. Image Tag ()

Image Syntax: Void element embedding raster or vector images into an HTML document.

```

```

Format	Full Form	Key Strengths / Primary Use Case
JPEG / JPG	Joint Photographic Experts Group	Lossy compression; ideal for photographs with complex color gradients
PNG	Portable Network Graphics	Lossless compression; supports sharp lines and alpha channel transparency
GIF	Graphics Interchange Format	Supports simple animations; limited to 256 indexed colors
SVG	Scalable Vector Graphics	XML vector graphics; scales infinitely without pixelation; ideal for logos/icons
WebP	Web Picture format	Modern Google format offering superior lossy/lossless compression

10. Table Tag (<table>)

Table Elements: Constructs structured data grids consisting of rows (<tr>), headers (<th>), and data cells (<td>).

- **colspan='N':** Merges cells horizontally across multiple columns.
- **rowspan='N':** Merges cells vertically across multiple rows.

```
<table border="1" cellpadding="6" cellspacing="0">
  <caption><strong>Examination Schedule</strong></caption>
  <thead>
    <tr>
      <th rowspan="2">Day</th>
      <th colspan="2">Sessions</th>
    </tr>
    <tr>
      <th>Morning</th>
      <th>Afternoon</th>
    </tr>
  </thead>
  <tbody>
```

```
<tr>
  <td>Monday</td>
  <td>Web Technologies</td>
  <td>Database Systems</td>
</tr>
<tr>
  <td>Tuesday</td>
  <td colspan="2" align="center">Project Lab Viva (Full Day)</td>
</tr>
</tbody>
<tfoot>
  <tr>
    <td colspan="3">Note: Hall tickets mandatory.</td>
  </tr>
</tfoot>
</table>
```

11. List Tags (, , <dl>)

- **Ordered List ():** Numbered sequence of items (). Attributes: type ('1', 'A', 'a', 'I', 'i'), start, reversed.
- **Unordered List ():** Bulleted list of items (). CSS list-style-type values: disc, circle, square, none.
- **Description List (<dl>):** Name-value glossary list comprising Terms (<dt>) and Definitions (<dd>).

```
<!-- Ordered List -->
<ol type="1" start="1">
  <li>Download package</li>
  <li>Run installation</li>
</ol>

<!-- Unordered List -->
<ul>
  <li>HTML5</li>
  <li>CSS3</li>
</ul>

<!-- Description List -->
<dl>
  <dt>HTTP</dt>
  <dd>Hypertext Transfer Protocol for web communication.</dd>
</dl>
```

12. Frame Tag (Historical) and Inline Frame (<iframe>)

Historical Frames: Historical HTML 4.01 <frameset> divided the browser window into separate sub-windows, each loading an independent HTML document. Deprecated in HTML5 due to broken bookmarks, poor SEO, accessibility failures, and security vulnerabilities.

Modern Standard: Modern HTML5 replaces framesets with the Inline Frame (<iframe>), embedding external documents inside the standard <body> flow.

```
<iframe src="https://www.example.com" width="600" height="350" title="Embedded Demo"
loading="lazy" sandbox="allow-scripts allow-same-origin"></iframe>
```

13. DIV Tag and SPAN Tag

Feature	<div> (Division)	 (Span)
Display Model	Block-Level element	Inline element
Line Breaking	Always begins on a new line	Stays inline with surrounding text flow
Width Occupied	Expands to 100% of parent container width	Occupies only width of inner content
Box Model	Full margin and padding support	Vertical margins do not displace adjacent text lines
Primary Use	Grouping major layout sections and cards	Styling specific words/phrases inside text

14. HTML Forms and 15. Form Input Types

Form Overview: HTML forms capture user input and submit data to server endpoints via HTTP requests.

Feature	GET Method	POST Method
Data Placement	Appended visibly to URL query string (?k=v)	Enclosed inside HTTP Request Body
Security	Insecure (Passwords exposed in URL / logs)	Secure for sensitive input (especially over HTTPS)
Payload Limit	Restricted by browser URL length (~2048 chars)	Virtually unlimited payload size
Caching / Bookmark	Can be cached and bookmarked	Cannot be cached or bookmarked
Best Used For	Search queries, idempotent read operations	Logins, registrations, file uploads, transactions

HTML Input Types: Comprehensive Input Types Demonstration:

- **type='text':** Single-line plain text field.
- **type='password':** Masked character input field preventing screen visibility.
- **type='email':** Validates email syntax (user@domain.com) upon submission.
- **type='number':** Restricts input to numbers with min, max, and step controls.
- **type='radio':** Mutually exclusive choice from a group. All options in a group **MUST** share identical name attribute.
- **type='checkbox':** Independent multiple selections allowed.
- **type='date' & type='time':** Native calendar date picker and clock time selector.
- **type='file':** File picker. Requires form attributes enctype='multipart/form-data' and method='POST'.
- **type='hidden':** Invisible data field passing tokens/IDs without user display.
- **type='submit':** Triggers form submission to server action URL.
- **type='reset':** Clears all form controls to default state.
- **type='button':** Generic clickable button wired to JavaScript events.

- **type='image'**: Graphical submit button transmitting click X/Y coordinates.

16. Textarea, 17. Select, 18. Button, and 19. Complete Form

- **<textarea>**: Multi-line text input box with rows and cols attributes (requires closing </textarea>).
- **<select>**: Dropdown combo box with <option> choices grouped by <optgroup>.
- **<button>**: Clickable push button supporting rich nested HTML icons and formatting.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Registration Form</title>
</head>
<body>
  <form action="register.php" method="POST" enctype="multipart/form-data">
    <fieldset>
      <legend><strong>Student Registration</strong></legend>
      <p><label for="uname">Full Name:</label><br>
      <input type="text" id="uname" name="uname" placeholder="John Doe" required></p>

      <p><label for="uemail">Email:</label><br>
      <input type="email" id="uemail" name="uemail" required></p>

      <p><label for="upass">Password:</label><br>
      <input type="password" id="upass" name="upass" minlength="8" required></p>

      <p>Gender:<br>
      <input type="radio" id="gm" name="gender" value="m" required><label
for="gm">Male</label>
      <input type="radio" id="gf" name="gender" value="f"><label
for="gf">Female</label></p>

      <p><label for="dept">Department:</label><br>
      <select id="dept" name="dept">
        <optgroup label="Engineering">
          <option value="cs">Computer Science</option>
          <option value="it">Information Technology</option>
        </optgroup>
      </select></p>

      <p><label for="resume">Upload Resume (PDF):</label><br>
      <input type="file" id="resume" name="resume" accept=".pdf" required></p>
```

```
<p><label for="comments">Address:</label><br>
<textarea id="comments" name="comments" rows="3" cols="35"></textarea></p>

<input type="hidden" name="token" value="sec_token_987">
<button type="submit">Register Now</button>
<button type="reset">Clear Form</button>
</fieldset>
</form>
</body>
</html>
```

Important Definitions — Unit III

- HTML: HyperText Markup Language defining web page structure and semantics.
- HTML Element: Complete unit consisting of opening tag, attributes, content, and closing tag.
- Void Element: Tag that does not contain content or closing tag (
, , <input>).
- DOM (Document Object Model): Hierarchical tree representation of HTML elements parsed by browsers.
- Block-Level Element: Starts on a new line and occupies 100% available container width (<div>, <p>).
- Inline Element: Renders within existing line flow without line breaks (, <a>,).
- HTML Form: Section containing interactive controls capturing user data for server submission.
- POST Method: Submits form data securely inside HTTP request body without URL size limits.
- Iframe: Inline frame embedding external HTML documents inside modern HTML5 pages.

Quick Revision — Unit III

- Document skeleton: <!DOCTYPE html> -> <html> -> <head> (metadata) + <body> (visible content).
- Tag vs Element vs Attribute: <p> is tag; <p class='x'>text</p> is element; class='x' is attribute.
- Semantic layout: <header>, <nav>, <main>, <section>, <article>, <aside>, <footer>.
- Hyperlinks: with absolute/relative URLs and #id anchors.
- Images: (alt is required for accessibility).
- Tables: <table> -> <thead>/<tbody>/<tfoot> -> <tr> (row) -> <th> (header) / <td> (data); rowspan/colspan.
- Lists: (numbered), (bulleted), <dl> (description with <dt> term and <dd> definition).
- Historical <frameset> is deprecated in HTML5; replaced by modern <iframe>.
- <div> is block-level layout container; is inline phrase wrapper.
- Forms: Radio buttons share name; Checkboxes allow multiple; Files require enctype='multipart/form-data'.

UNIT IV

1. Introduction to CSS

Definition: Cascading Style Sheets (CSS) is a declarative stylesheet language used to describe the presentation, visual styling, typography, and responsive layout of documents written in HTML or XML.

Purpose & Role: CSS was designed to separate document content/structure (written in HTML) from document presentation (visual styling, grid positioning, animations). This decoupling ensures maintainability and site-wide visual consistency.

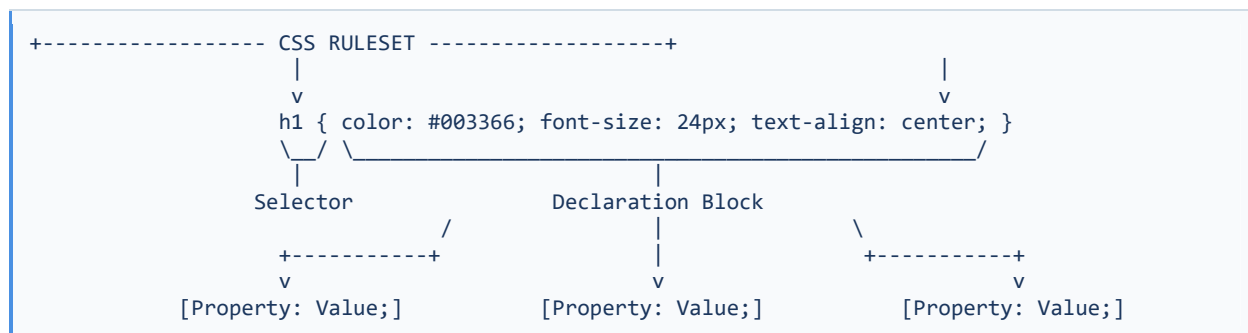
Key Advantages: Major Advantages of CSS:

- **Site-Wide Consistency & Reusability:** Changing a single external CSS file updates the look and feel of thousands of web pages simultaneously.

- **Faster Page Loading:** External CSS files are cached by web browsers upon initial download, drastically reducing bandwidth consumption and server load.
- **Responsive Multi-Device Layouts:** CSS Media Queries (@media) allow responsive adaptation across mobile phones, tablets, high-resolution desktops, and print media.
- **Improved Accessibility & SEO:** Decoupling styles leaves HTML clean, semantic, and easily interpretable by search engine crawlers and screen readers.

2. CSS Syntax

Rule Structure: A CSS ruleset consists of a Selector and a Declaration Block.



- **Selector:** The HTML element, class, ID, or pattern targeted for styling (e.g., h1, .btn, #header).
- **Declaration Block:** Enclosed in curly braces { }; contains one or more style declarations separated by semicolons.
- **Property:** A style feature to be modified (e.g., color, font-size, margin).
- **Value:** The specific configuration assigned to the property (e.g., red, 16px, 10px auto).

3. CSS Selectors

Selector Types: CSS selectors define which HTML elements are targeted by a ruleset.

Selector Type	Syntax Example	Matching Criteria
Universal Selector	* { margin: 0; }	Targets every single element in the DOM tree.
Type / Element Selector	p { color: #333; }	Targets all instances of specified HTML tag (<p>).
Class Selector	.card { padding: 15px; }	Targets all elements with class='card' (preceded by dot .).
ID Selector	#main-nav { top: 0; }	Targets single unique element with id='main-nav' (preceded by #).
Grouping Selector	h1, h2, h3 { color: navy; }	Applies identical styles to multiple comma-separated selectors.
Descendant Combinator	div p { font-size: 14px; }	Targets <p> anywhere inside <div> (nested at any depth).
Child Combinator	ul > li { list-style: square; }	Targets that are direct children of .
Adjacent Sibling	h1 + p { font-weight: bold; }	Targets <p> immediately following

		<h1>.
General Sibling	h1 ~ p { color: gray; }	Targets all <p> siblings that follow <h1>.
Attribute Selector	input[type='text'] { border: 1px; }	Targets elements possessing specific attribute values.
Pseudo-Classes	a:hover, li:first-child	Styles elements based on state or DOM position.
Pseudo-Elements	p::first-letter, div::after	Styles specific sub-parts of an element or injects content.

Specificity Hierarchy (Cascade Resolution)

Specificity Rules: When multiple conflicting CSS rules target the exact same HTML element, the browser calculates specificity to determine which rule wins:

- **1. Inline Styles (style='...'):** Assigned weight of 1000 points (highest standard priority).
- **2. ID Selectors (#id):** Assigned weight of 100 points.
- **3. Class, Attribute, and Pseudo-Class Selectors (.class, [type], :hover):** Assigned weight of 10 points.
- **4. Element and Pseudo-Element Selectors (p, div, ::before):** Assigned weight of 1 point.
- **5. Universal Selector (*):** Assigned weight of 0 points.
- **!important Rule:** Overrides all normal specificity calculations (use sparingly for emergency overrides).

4. Embedding CSS to HTML (Inline, Internal, External)

Three Embedding Methods: There are three primary methods to attach CSS stylesheets to HTML documents.

A. Inline Styles

Syntax & Use: Written directly inside the HTML element's opening tag using the style attribute.

```
<p style="color: blue; font-size: 16px; margin-bottom: 10px;">This is styled with inline CSS.</p>
```

- **Pros:** Highest specificity; useful for quick testing or dynamic server-generated styling.
- **Cons:** Mixes content with presentation; violates DRY (Don't Repeat Yourself); impossible to maintain site-wide.

B. Internal (Embedded) Style Sheets

Syntax & Use: Written inside the <style> tag located within the <head> section of a single HTML document.

```
<head>
  <style>
    body { font-family: Arial, sans-serif; background-color: #f8f9fa; }
    .highlight { color: #d9534f; font-weight: bold; }
  </style>
</head>
```

- **Pros:** Styles entire single page without external HTTP requests.
- **Cons:** Styles cannot be shared across other HTML files on the website.

C. External Style Sheets

Syntax & Use: Written in an independent plain text file with a .css extension (e.g., styles.css) and linked in the HTML <head> using <link>.

```
<head>
  <!-- Linking external CSS file -->
  <link rel="stylesheet" type="text/css" href="css/styles.css">
</head>
```

- **Pros:** Complete separation of concerns; shared across entire website; cached by browser for maximum speed.

Feature	Inline Styles	Internal Style Sheet	External Style Sheet
Location	Inside HTML tag (style='...')	Inside <head> using <style>	Separate external .css file
Scope	Single specific element	Single HTML document	Entire website (all linked pages)
Maintenance	Extremely poor (Scattered)	Moderate (Per-page)	Excellent (Centralized stylesheet)
Browser Caching	No caching possible	No separate caching	Cached efficiently by browser
Specificity	Highest (1000 points)	Standard selector specificity	Standard selector specificity

5. Formatting Fonts

Font Properties: CSS provides extensive font properties to control typeface family, size, weight, style, and web font loading.

Property	Description	Permitted Values / Examples
font-family	Defines prioritized list of font family names (Font Stack)	font-family: 'Helvetica Neue', Arial, sans-serif;
font-size	Sets text dimensions (Absolute: px, pt; Relative: em, rem, %)	font-size: 16px; font-size: 1.25rem;
font-weight	Controls character stroke boldness	font-weight: normal; font-weight: bold; font-weight: 700;
font-style	Sets italic or oblique text styling	font-style: normal; font-style: italic;
font-variant	Renders lowercase text into small uppercase glyphs	font-variant: small-caps;
font (Shorthand)	Combines style, weight, size/line-height, family in one line	font: italic bold 16px/1.5 Arial, sans-serif;

Web Fonts and @font-face

@font-face Rule: Enables web designers to load custom web fonts from remote servers or Google Fonts rather than relying only on local system fonts.

```
@font-face {  
  font-family: 'CustomUniversityFont';  
  src: url('fonts/custom-font.woff2') format('woff2'),  
       url('fonts/custom-font.woff') format('woff');  
  font-weight: normal;  
  font-style: normal;  
}  
body { font-family: 'CustomUniversityFont', sans-serif; }
```

6. Text and Background Properties

A. CSS Color Formats

- **Named Colors:** Predefined keywords (e.g., red, navy, teal).
- **Hexadecimal (Hex):** 6-digit or 3-digit hexadecimal notation (e.g., #1B365D, #FFF).
- **RGB Format:** Red, Green, Blue channel integers 0–255 (e.g., rgb(27, 54, 93)).
- **RGBA Format:** RGB with Alpha channel transparency 0.0–1.0 (e.g., rgba(27, 54, 93, 0.85)).
- **HSL Format:** Hue (0-360), Saturation (0-100%), Lightness (0-100%) (e.g., hsl(215, 55%, 24%)).

B. Text Formatting Properties

Property	Description	Code Example
color	Sets foreground text color	color: #1B365D;
text-align	Controls horizontal text alignment (left, right, center, justify)	text-align: justify;
text-decoration	Adds or removes line decorations (none, underline, line-through)	text-decoration: underline;
text-transform	Controls character capitalization (uppercase, lowercase, capitalize)	text-transform: uppercase;
line-height	Sets vertical distance between lines of text (Leading)	line-height: 1.6;
letter-spacing	Adjusts horizontal spacing between individual characters (Tracking)	letter-spacing: 1.5px;
text-shadow	Applies drop shadow to text (h-offset v-offset blur color)	text-shadow: 2px 2px 4px rgba(0,0,0,0.4);

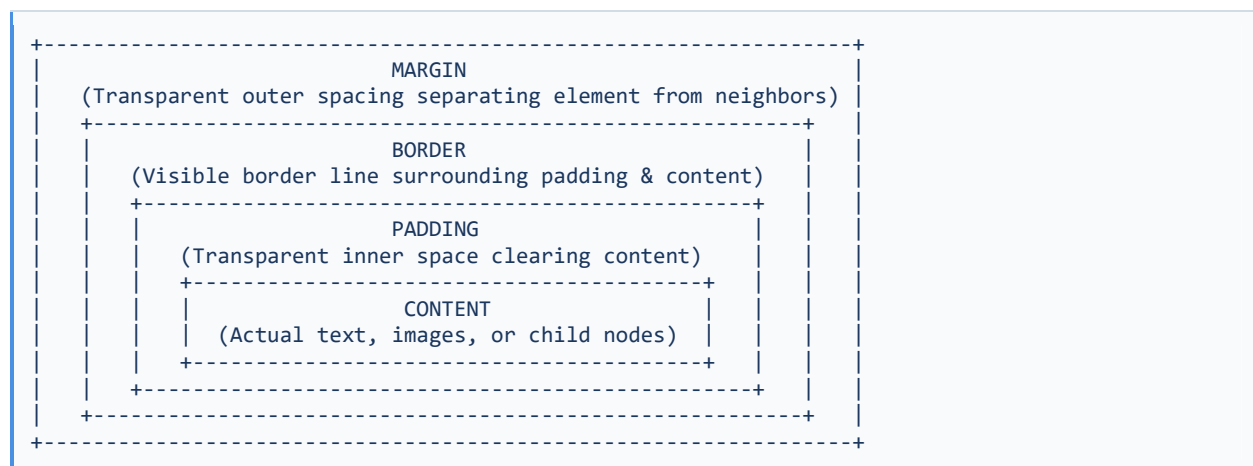
C. Background Properties

Property	Description	Code Example
----------	-------------	--------------

background-color	Sets solid color fill for element background	background-color: #F0F4F8;
background-image	Sets one or more background images or gradients	background-image: url('bg.png');
background-repeat	Controls background tiling (repeat, no-repeat, repeat-x, repeat-y)	background-repeat: no-repeat;
background-position	Sets initial origin position of background image	background-position: center top;
background-size	Controls scaling (cover: fills box, contain: fits box, auto)	background-size: cover;
background-attachment	Specifies if image scrolls with page or stays fixed	background-attachment: fixed;
background (Shorthand)	Combines color, image, position, size, repeat in one line	background: #fff url('bg.jpg') no-repeat center/cover;

7. Borders and the CSS Box Model

Box Model Definition: The CSS Box Model is the foundational layout paradigm where every rendered HTML element is treated as a rectangular box wrapped in four distinct layers.



- **1. Content:** The innermost area where actual text, images, or media reside. Dimensions set via width and height.
- **2. Padding:** The transparent clearance area immediately surrounding the content, inside the border. Inherits background color.
- **3. Border:** The line encircling the padding and content. Styled via border-width, border-style, border-color, and border-radius.
- **4. Margin:** The outermost transparent area separating this element from adjacent sibling elements. Can collapse vertically (Margin Collapse).

Border Properties

- **border-style:** Controls line style: solid, dashed, dotted, double, groove, ridge, inset, outset, none.
- **border-width:** Sets border thickness (e.g., 1px, 3px, thin, medium, thick).

- **border-color:** Sets border color (e.g., #1B365D, transparent).
- **border-radius:** Rounds element corners into curves or circles (e.g., border-radius: 8px; border-radius: 50%;).
- **outline vs border:** Draws an accent line outside the border that does NOT occupy space in the box model layout.

The `box-sizing` Property

Value	Calculation Formula	Behavior / Impact
content-box (Default)	Total Width = width + padding + border + margin	Adding padding expands visual box size outward, breaking fixed grid layouts.
border-box (Modern)	Total Width = width (Padding & border absorbed inside)	Padding and border are drawn inside declared width. Modern industry standard.

```
/* Modern CSS Box-Sizing Reset */
*, *::before, *::after {
  box-sizing: border-box;
  margin: 0;
  padding: 0;
}
```

Important Definitions — Unit IV

- CSS (Cascading Style Sheets): Declarative stylesheet language formatting HTML/XML presentation.
- CSS Specificity: Numeric weighting algorithm resolving conflicting style rules (Inline > ID > Class > Element).
- Box Model: Rectangular layout model consisting of Content, Padding, Border, and Margin.
- Padding: Inner transparent space between content and border.
- Margin: Outer transparent space separating an element from neighboring elements.
- box-sizing: border-box: Layout calculation mode including padding and border inside the declared width/height.
- External Stylesheet: Separate .css file linked via <link rel='stylesheet'> enabling global site styling.

Quick Revision — Unit IV

- CSS separates structure (HTML) from visual presentation (CSS).
- Ruleset = Selector + Declaration Block { property: value; }.
- 3 Embedding Methods: Inline (style='...'), Internal (<style>), External (<link href='...'>).
- Box Model 4 Layers: Content (inner) -> Padding -> Border -> Margin (outer).
- Border properties: border-width, border-style, border-color, border-radius.
- Colors: Hex (#1B365D), RGB/RGBA (Alpha transparency), HSL.
- Always use `box-sizing: border-box` to prevent padding from altering element widths.

UNIT V

1. Introduction to XML

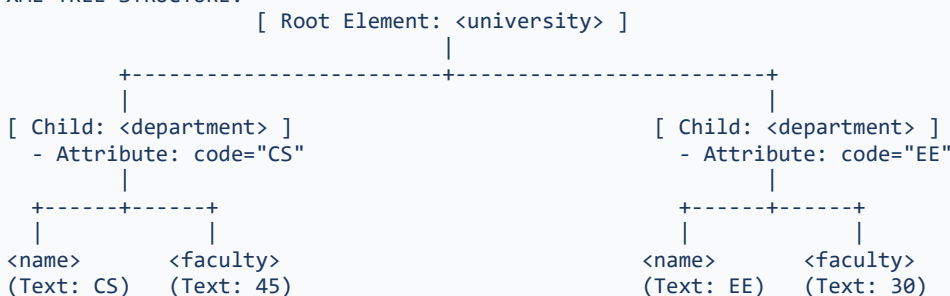
Definition: Extensible Markup Language (XML) is a text-based, self-describing, software- and hardware-independent markup language standardized by the W3C (1998) designed to store, describe, structure, and transport data.

Core Purpose: Unlike HTML, XML is not designed to display data visually. XML is a pure data description format that allows diverse computing platforms, databases, and programming languages to exchange structured information seamlessly.

Major Features of XML: Key Characteristics:

- **Self-Descriptive & Extensible:** Tags are not predefined; authors create meaningful custom domain tags (e.g., <student>, <gpa>, <transaction>).
- **Strict Parsing Rules:** Strict syntactic rules enforce well-formedness; parsers halt on syntax errors.
- **Hierarchical Tree Model:** Stores data in a hierarchical parent-child tree structure.
- **Platform & Vendor Independent:** Acts as a neutral bridge between incompatible databases and back-end systems.

XML TREE STRUCTURE:



```

<?xml version="1.0" encoding="UTF-8"?>
<!-- XML Prologue / Declaration -->
<university>
  <department code="CS">
    <name>Computer Science</name>
    <faculty>45</faculty>
  </department>
  <department code="EE">
    <name>Electrical Engineering</name>
    <faculty>30</faculty>
  </department>
</university>
  
```

2. Difference between HTML and XML

Feature / Parameter	HTML (HyperText Markup Language)	XML (Extensible Markup Language)
Primary Goal	Display and format data for human viewing	Store, describe, and transport data for machines
Tag Predefinition	Tags are fixed & predefined (<h1>, <p>, <a>)	Tags are completely custom and user-defined

Syntax Strictness	Lenient (Browsers tolerate missing end tags)	Extremely strict (Malformed syntax stops parser)
Case Sensitivity	Case-insensitive (<p> equals <P>)	Strictly case-sensitive (<Book> != <book>)
Closing Tags	Optional on void elements (,)	Mandatory for every single element (<tag/>)
Data vs Presentation	Combines data with visual layout presentation	Pure data storage; contains zero styling rules
Whitespace Handling	Collapses multiple consecutive spaces into one	Preserves exact whitespace inside elements
Schema Validation	No structural schema validation	Validates against formal DTD or XML Schema (XSD)

3. XML Editors

- **General Code Editors:** General code editors with XML plugins providing syntax coloring, tag matching, and formatting. Examples: Visual Studio Code, Notepad++, Sublime Text.
- **Specialized XML IDEs:** Dedicated enterprise XML IDEs featuring visual schema designers, DTD validation, XSLT debuggers, and XPath evaluators. Examples: Altova XMLSpy, Oxygen XML Editor.
- **Core Editor Features:** Real-time syntax linting, DTD/XSD schema validation, auto-closing tag generation, tree navigation, and XPath query execution.

4. XML Elements and Attributes

- **XML Elements:** The building blocks of XML consisting of an opening tag, content, and closing tag (<title>Web Tech</title>) or self-closing void tag (<spacer/>).
- **Element Naming Rules:** Names are case-sensitive; must start with a letter or underscore; cannot start with numbers, punctuation, or 'xml'; cannot contain spaces.
- **XML Attributes:** Name-value metadata pairs attached inside opening tags (<book id='b101'>). Values MUST be enclosed in quotes.

Feature	XML Elements	XML Attributes
Purpose	Used to represent core data and structural entities	Used to represent metadata or descriptive properties
Multiple Values	Can easily contain multiple child elements & lists	Cannot contain multiple values cleanly
Extensibility	Highly extensible; can add child nodes later	Hard to extend; cannot contain child structures
Tree Hierarchy	Forms the hierarchical DOM tree nodes	Attached as properties on element nodes
Best Practice Rule	Store information intended for reading/processing in elements;	

store unique IDs and data types in attributes.

5. XML Document Type Definition (DTD)

Definition: A Document Type Definition (DTD) defines the legal building blocks, grammatical rules, allowed elements, and attribute constraints for an XML document.

Well-Formed vs. Valid XML: Two Classes of XML Documents:

- **Well-Formed XML:** Conforms to fundamental XML syntax rules (has root element, proper nesting, closed tags, quoted attributes).
- **Valid XML:** Is Well-Formed AND conforms to an attached grammatical schema definition (DTD or XSD).

Internal vs. External DTD

```
<!-- INTERNAL DTD EXAMPLE -->
<!DOCTYPE bookstore [
  <!ELEMENT bookstore (book+)>
  <!ELEMENT book (title, author, price)>
  <!ATTLIST book id ID #REQUIRED category (tech|fiction) "tech">
  <!ELEMENT title (#PCDATA)>
  <!ELEMENT author (#PCDATA)>
  <!ELEMENT price (#PCDATA)>
]>
<bookstore>
  <book id="b1" category="tech">
    <title>Web Technologies</title>
    <author>Prof. Smith</author>
    <price>49.99</price>
  </book>
</bookstore>
```

External DTD Declaration: External DTD Syntax: `<!DOCTYPE bookstore SYSTEM 'bookstore.dtd'>` (Private/System) or `<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0//EN" 'xhtml.dtd'>` (Public).

DTD Element / Symbol	Meaning / Cardinality Rule
#PCDATA	Parsed Character Data (Text that will be parsed for tags).
, (Comma)	Strict sequential order (e.g., (title, author) requires title then author).
(Pipe)	Choice / OR operator (e.g., (card cash) permits either card or cash).
+ (Plus)	One or more occurrences mandatory (at least once).
* (Asterisk)	Zero or more occurrences allowed (optional repeated).
? (Question Mark)	Zero or one occurrence allowed (optional single).
#REQUIRED	Attribute is mandatory on the element.
#IMPLIED	Attribute is optional on the element.

#FIXED 'val'

Attribute value is fixed and unchangeable.

- **Limitations of DTD:** Does not support modern data types (treats almost all data as strings; cannot enforce numbers, dates, or regex patterns); not written in XML syntax; lacks namespace support.

6. XML Schema Definition (XSD)

Definition: XML Schema (XSD) is an XML-based, W3C-standard schema language used to define the structure, content, data types, and validation rules of XML documents.

Feature	XML DTD	XML Schema (XSD)
Syntax	Non-XML proprietary DTD syntax	Written in standard XML syntax (Uses XML tags)
Data Typing	Extremely limited (#PCDATA, CDATA strings)	Rich built-in datatypes (string, integer, date, boolean)
Namespace Support	No support for XML namespaces	Full native XML namespace support
Extensibility	Rigid; difficult to extend or inherit	Extensible; supports object-oriented type inheritance
Custom Constraints	Cannot enforce ranges or regex patterns	Allows custom facets (minInclusive, maxInclusive, regex)

Complete XML Schema (XSD) Example

```
<?xml version="1.0" encoding="UTF-8"?>
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
  <xs:element name="student">
    <xs:complexType>
      <xs:sequence>
        <xs:element name="name" type="xs:string"/>
        <xs:element name="age">
          <xs:simpleType>
            <xs:restriction base="xs:integer">
              <xs:minInclusive value="18"/>
              <xs:maxInclusive value="65"/>
            </xs:restriction>
          </xs:simpleType>
        </xs:element>
        <xs:element name="email" type="xs:string"/>
      </xs:sequence>
      <xs:attribute name="rollno" type="xs:string" use="required"/>
    </xs:complexType>
  </xs:element>
</xs:schema>
```

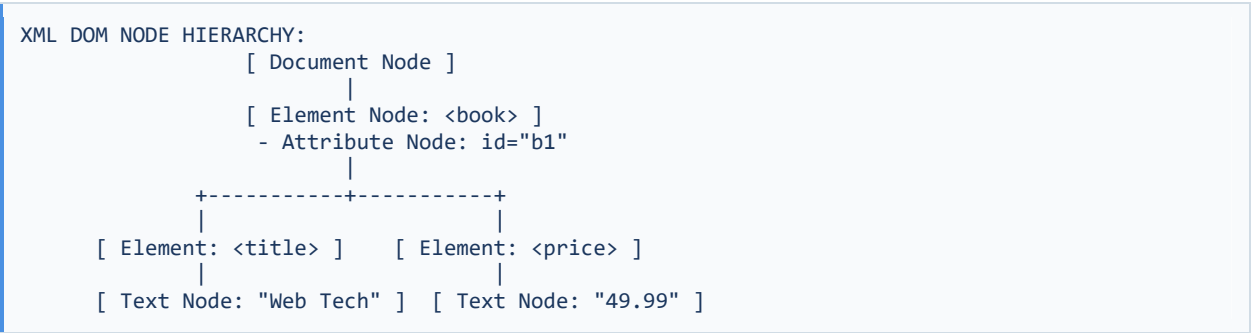
7. XML Parsers

Definition: An XML Parser is a software library or module that reads an XML document, verifies that it is Well-Formed (and optionally Valid), and exposes its data structure via programmatic APIs.

Feature	DOM Parser (Document Object Model)	SAX Parser (Simple API for XML)
Parsing Model	Tree-Based Parser (In-Memory Tree)	Event-Driven Stream Parser (Push Parser)
Memory Usage	High; loads entire XML tree into RAM	Extremely low; processes node stream on-the-fly
Navigation	Bidirectional; random access to any node	Forward-only; sequential one-pass reading
Modification	Allows reading, updating, adding, deleting nodes	Read-only; cannot modify XML document structure
Performance	Slower on very large files (e.g., 500MB+)	Extremely fast; ideal for massive streaming data
Best Used For	Interactive XML editing, XPath search, small/medium files	Large XML feeds, logging streams, memory-constrained devices

8. XML Document Object Model (DOM)

Definition: The XML Document Object Model (XML DOM) is a standard W3C programming interface that represents an XML document as a tree of objects (nodes) accessible by programming languages like JavaScript, Python, and Java.



DOM Property / Method	Description / Function
documentElement	Returns the root element of the XML document.
getElementsByTagName(tag)	Returns NodeList array of all matching element nodes.
parentNode / childNodes	Navigates to parent node or returns list of child nodes.
firstChild / lastChild	Accesses first or last child node of target element.

nodeName / nodeValue	Returns name of node or raw text value of node.
getAttribute(name)	Retrieves string value of specified attribute.
createElement(tag) / appendChild()	Creates a new element node and attaches it to parent tree.

Client-Side JavaScript XML DOM Parsing Example

```
// JavaScript XML DOM Parsing
const xmlText = `<title>Web Technologies</title><author>Dr. Kumar</author></book>`;
const parser = new DOMParser();
const xmlDoc = parser.parseFromString(xmlText, "text/xml");

// Accessing DOM nodes
const titleText = xmlDoc.getElementsByTagName("title")[0].childNodes[0].nodeValue;
console.log("Book Title: " + titleText); // Outputs: Web Technologies
```

Important Definitions — Unit V

- XML (Extensible Markup Language): W3C markup language designed to store, describe, and transport data.
- Well-Formed XML: XML document complying with core syntax rules (closed tags, single root, proper nesting).
- Valid XML: Well-formed XML document that strictly conforms to an attached DTD or XSD schema.
- DTD (Document Type Definition): Grammatical schema defining allowed elements and attributes in an XML document.
- XML Schema (XSD): W3C XML-based schema language supporting rich data types and namespace constraints.
- DOM Parser: Tree-based parser loading entire XML document into RAM for random access and editing.
- SAX Parser: Event-driven streaming parser processing XML sequentially with minimal memory overhead.
- XML DOM: Standard tree-structured programming interface representing XML elements as navigable objects.

Quick Revision — Unit V

- HTML displays data (Presentation); XML transports data (Structure & Storage).
- XML is strictly case-sensitive, requires closing tags, and mandates a single root element.
- Well-Formed = Syntax correct; Valid = Conforms to DTD or XSD Schema.
- XSD is superior to DTD because it is written in XML and supports data types (string, integer, date).
- DOM Parser = In-memory tree, random access, high RAM; SAX Parser = Event stream, fast, low RAM.
- XML DOM represents documents as Document -> Element -> Attribute -> Text nodes.